CookMem Tutorial 6

Replacing malloc and new

What a global allocator gains, and what can go wrong



For anyone considering a different allocator for a whole program

Why replace the allocator?

1

No code changes

Every allocation changes at once, including those in libraries.

2

Allocators built for it

jemalloc, mimalloc and tcmalloc replace malloc, thread-safely.

3

CookMem is explicit

Contexts are used by one thread at a time, for a job you choose.

CookMem's performance tests replace malloc to run the same programs on dlmalloc and on CookMem. Under AddressSanitizer, which replaces malloc too, they crash.

Four levels of replacement



Broader

fewer code changes, more that can go wrong

Narrower

more code to touch, more control

Replacing malloc and free

```
extern "C" void* malloc (std::size_t size)
{
    return threadContext ().allocate (size);
}

extern "C" void free (void* ptr)
{
    threadContext ().deallocate (ptr);  // which context owns ptr?
}

// ... and calloc, realloc, aligned_alloc, posix_memalign,
// memalign, valloc, pvalloc, malloc_usable_size, ...
```

a sketch, not a complete replacement

Linux
Define the functions, or LD_PRELOAD.
Replace the whole family together.

Windows
DLLs on the shared C runtime keep its heap;
a redirection DLL patches it.

macOS
Allocators register malloc zones.

It is global: every library in the process, and with LD_PRELOAD every child process, gets the new allocator.
A missing function corrupts the heap: memory allocated by one allocator is freed by the other.

Replacing operator new and delete

```
void* operator new (std::size_t size)
{
    void* ptr = threadContext ().allocate (size);
    if (ptr == nullptr)
        throw std::bad_alloc ();
    return ptr;
}

void operator delete (void* ptr) noexcept
{
    threadContext ().deallocate (ptr);
}

// ... and 18 more: arrays, nothrow, sized, aligned
```

a sketch, not a complete replacement

Functions to replace

20

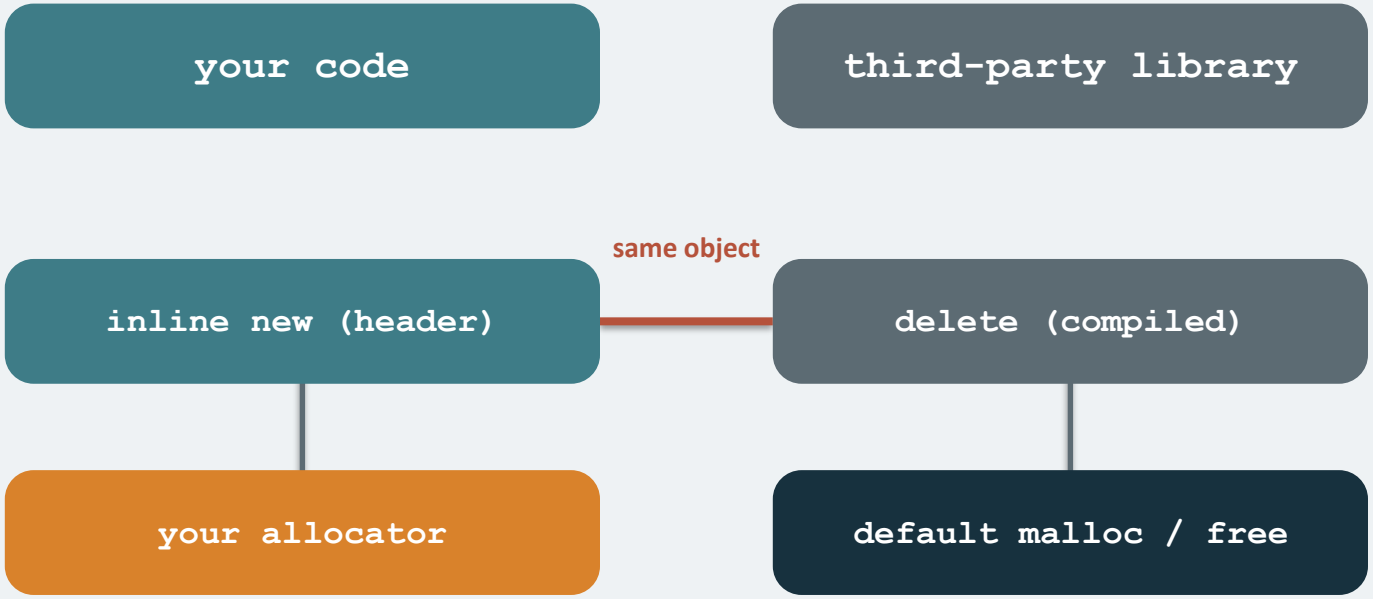
8 operator new
12 operator delete
in C++17

Not covered: malloc, so C libraries keep the system allocator.

Many details, but it can be worth it: it is standard, and it leaves C code alone.

Library boundaries and mismatched pairs

One object, two allocators



The object is freed by an allocator that did not allocate it: heap corruption.

DLL and shared library boundaries

A replacement in the program does not reach libraries that resolve new and delete themselves: every DLL on Windows, and on Linux libraries with a static C++ runtime or -Bsymbolic.

Inline halves of a pair

new inlined from a header uses your allocator; delete compiled into a library uses the default, or the other way around.

A class-specific operator new

```
struct Node
{
    static void* operator new (std::size_t size)
    {
        void* ptr = astContext ().allocate (size);
        if (ptr == nullptr)
            throw std::bad_alloc ();
        return ptr;
    }
    static void operator delete (void* ptr)
    {
        astContext ().deallocate (ptr);
    }
    virtual ~Node () { }
};
```

a sketch

One by one

`delete node;`

runs the destructor, then gives the memory back to the context

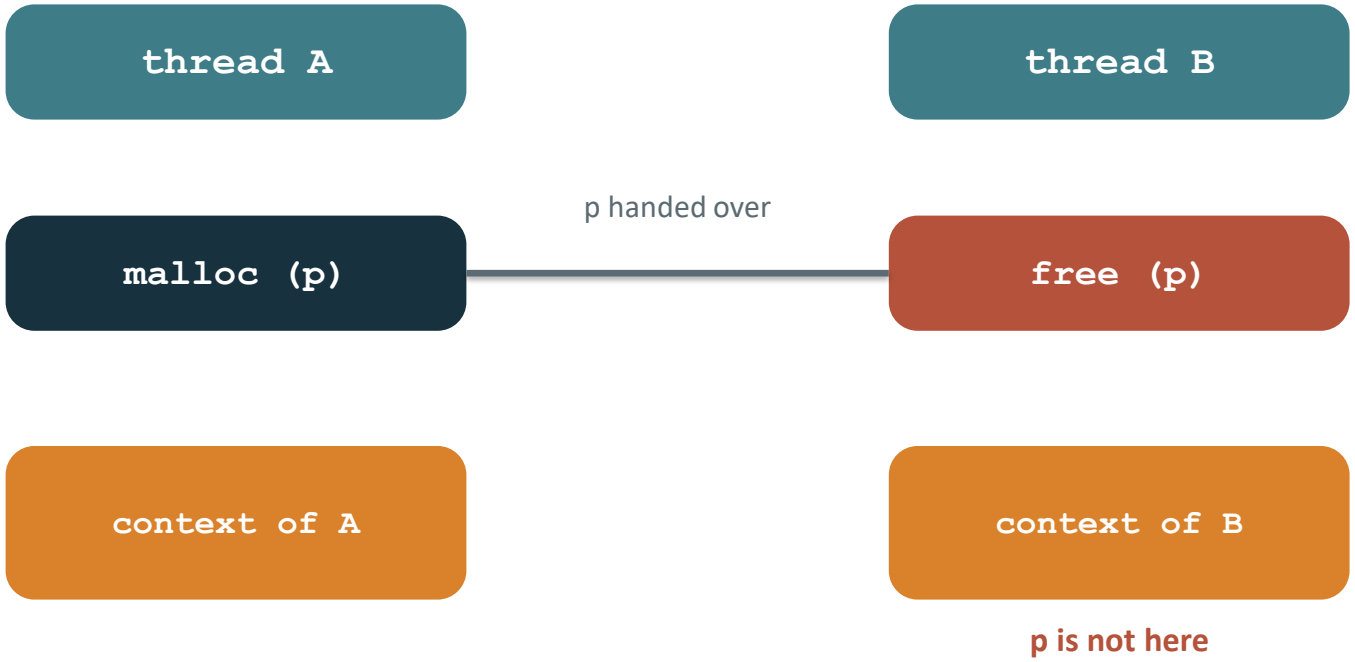
All at once

`astContext ().releaseAll ();`

no delete and no destructors: everything goes

RAII caveat: skip destructors only for objects that own nothing but context memory.

Threads: which context owns this pointer?

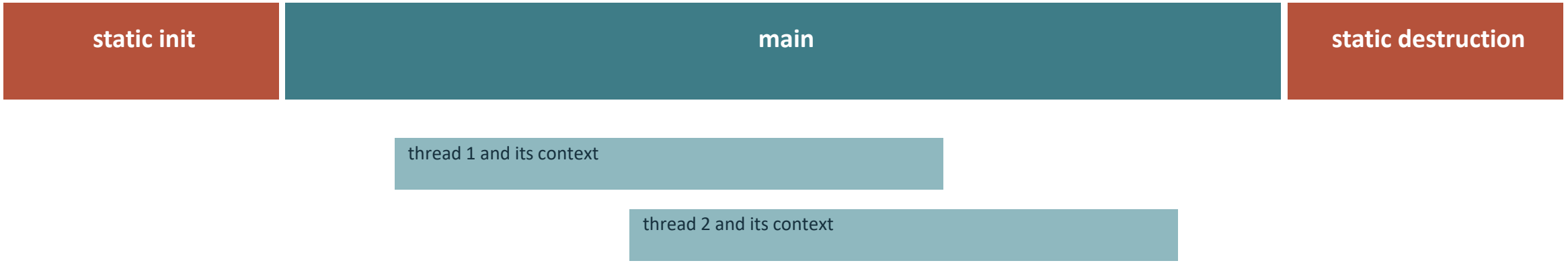


Find the owner
contains () checks every segment: slow on every free.

Free into it
Another thread's context needs a lock.

Or use a thread-safe allocator
jemalloc and mimalloc are built for this.

Startup and shutdown



Before main

Static objects allocate before any context is set up.

After a thread ends

Its context is gone, but its memory may still be freed.

After main

Static destructors free memory in an order that is hard to control.

Recursion and debugging tools

Recursion

The allocator allocates

A logger with a hash map, or printf in an error path, calls malloc again.

Keep it allocation-free

Use fixed buffers, or a separate allocator, inside the replacement.

Debugging tools

AddressSanitizer

Replaces malloc itself; another replacement conflicts with it.

Valgrind

Needs --soname-synonyms=somalloc=NONE, and sees segments, not allocations.

Releasing everything, globally

One global context: releaseAll () frees all of it

stream buffers	still in use
locale data	still in use
library caches	still in use
static strings	still in use
your objects	yours

Their next use touches freed memory.

Release in bulk only memory you own: an explicit context, or the context behind a class-specific operator new.

What to use when

Replace malloc	Global: every library and child process. Use jemalloc, mimalloc or tcmalloc.
Replace new and delete	Many details to get right, but standard; it can be worth the effort.
A job, a request, a thread	Safer: an explicit CookMem context, with Allocator or MemResource.
The objects of one graph	Safer: a class-specific operator new backed by a context.
A global CookMem replacement	Only with one thread, static linking, no memory crossing libraries, no releaseAll.

Summary

1. **Replacing malloc is global**
   Every library and child process, from before main to after it: use an allocator built for it.

2. **Replacing new and delete takes care**
   20 functions and library boundaries to get right, but the effort can be worth it.

3. **The safer approach: use CookMem explicitly**
   A context per job with Allocator or MemResource, released in one call.

